CHAPTER 23

Hotel Reservation System

Search rooms, reserve with no double booking, and handle concurrency and idempotency (Marriott-style).

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design the backend for a hotel chain's reservation system (e.g. Marriott: 5,000 hotels, 1 M rooms). Users search for hotels, view room types, **book a room for a date range**, cancel, and admins manage inventory. The core challenge is **preventing double booking** under concurrent requests and retries while staying available.

Why interviewers ask it: A transactional system design question: relational schema, concurrency control (locks / optimistic), idempotency and microservice consistency.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Scale?	5,000 hotels, 1 M rooms total.
Payment?	Assume payment is handled by an external gateway; reservation only.
Cancellations?	Supported.
Overbooking?	Allow up to 10% overbooking.
Traffic pattern?	Not high QPS but spikes in seasonal events; strong consistency matters.

3. Requirements

Functional

- Search hotels / room types / availability.
- Reserve a room type for a date range.
- Cancel a reservation.
- Admin CRUD for hotels, room types, inventory.
- Support overbooking by 10%.

Non-functional

- **No double booking** (high concurrency correctness).
- Moderate scalability.
- Idempotent booking calls.
- High availability for reads; consistency for writes.

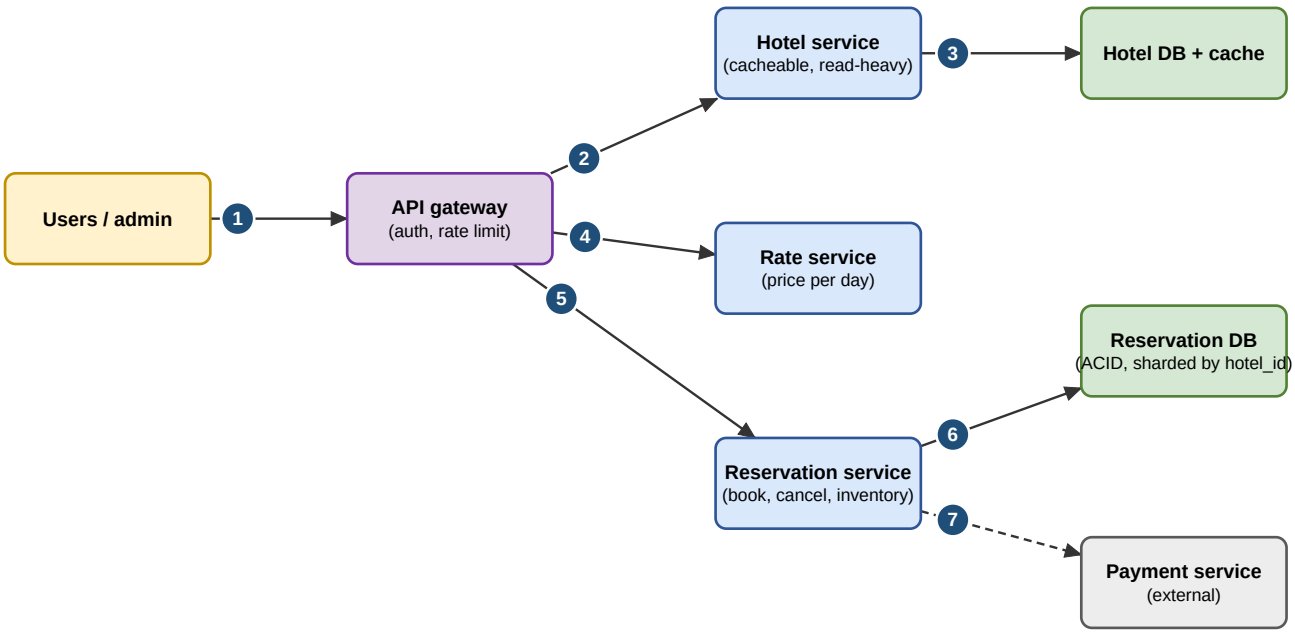
4. Back-of-the-envelope estimation

- 5,000 hotels x 200 rooms = 1 M rooms. Assume 70% occupancy and an average stay of 3 days -> about 240 K reservations/day = about **3 reservation QPS** (very low).
- Funnel: view hotel : view room : book = 100 : 10 : 1 -> hotel page about 300 QPS, order page about 30 QPS, **reservation about 3 QPS** (peak 10-100x for promos).
- Inventory table: 5,000 hotels x 20 room types x 2 years x 365 days = about 73 M rows - fits a single sharded relational DB.

5. API / data model

```
GET  /v1/hotels/{id}/rooms?checkin=..&checkout=..     POST /v1/reservations  { hotelId, roomTypeId, startDate, e
DELETE /v1/reservations/{id}      Admin: POST/PUT/DELETE /v1/hotels, /v1/hotels/{id}/rooms
Tables: hotel, room_type, room_type_inventory(hotel_id, room_type_id, date, total_inventory, total_reserved), re
```

6. High-level design



Step	What happens
1	Users and admins call the **API gateway**, which authenticates and rate-limits, then routes by path to the right service.
2	**Search and browse:** the hotel service returns hotel and room-type details (cacheable, read-heavy).
3	Hotel data is served from the hotel DB with a cache in front; it changes rarely.
4	The rate service returns the **price per night** for the requested dates.
5	**Book:** the client sends `POST /reservations` with a pre-generated `reservation_id` (the **idempotency key**).
6	In **one ACID transaction**, the reservation service checks `total_reserved + n <= total_inventory x 1.1` for every date, increments `total_reserved`, and inserts the reservation (a duplicate ID fails on the unique key). Sharded by `hotel_id`.
7	It then calls the external payment service. On success the status becomes paid; on failure or timeout, inventory is released (compensation).

Request flow

- Client opens the booking page -> server generates a unique **reservation ID** (idempotency key) -> user submits with that ID.
- Reservation service runs one DB transaction: check `total_reserved + requested <= total_inventory x 1.1` for every date in the range, increment `total_reserved`, insert the reservation (status pending). Commit.
- Payment succeeds -> status = paid. Failure/timeout -> release inventory (decrement) and mark cancelled.

7. Deep dive

Data model: inventory per room type per date

- Track availability by **room type and date**, not by room number (guests book a type; room number assigned at check-in). The reservation range updates each date row.
- `total_inventory - total_reserved` gives availability; overbooking allows `total_reserved <= total_inventory x 1.1`.

The concurrency problem

- **Race 1:** two users book the last room simultaneously. **Race 2:** one user double-clicks / retries the same request. Both must not result in two bookings.

Solving Race 1: concurrency control

- **Pessimistic lock:** `SELECT ... FOR UPDATE` on inventory rows; correct, simple, but blocks others and risks deadlocks; fine for low QPS.
- **Optimistic lock:** add a `version` column; `UPDATE ... WHERE version = ?`; retry on conflict. Good when contention is low, but many retries when hot.
- **Database constraints:** `CHECK (total_inventory - total_reserved >= 0)` makes the DB reject overbooking (as an extra safety net, works with the 10% rule by constraining total_reserved).
- For low QPS (3/s) all work; **pessimistic or constraint + transaction** is the simplest. Optimistic shines when reads dominate and conflicts are rare.

Solving Race 2: idempotency

- Client includes a unique `reservation_id` generated before submit; make it a **primary key / unique constraint** on the reservation table. A duplicate insert fails -> return the existing result. Disable the button client-side too, but the server guarantee is what matters.

Scaling the database

- Shard by `hotel_id` (or `hash(hotel_id) % N`): each hotel's inventory lives in one shard, so booking is a **single-shard transaction**. Cache availability for search (`Redis`) with TTL; the DB is the source of truth. Read replicas for hotel/room info.
- **Inventory cache pattern:** search reads from Redis (eventual); booking decrements in the DB then updates the cache (or invalidates it). Cache is advisory only.

Microservices and distributed transactions

- If hotel, rate, reservation, payment are separate services/databases, a booking spans services -> need **Two-Phase Commit** (blocking, complex) or **Saga** with compensating actions (release inventory on payment failure), or an **event-driven** flow with idempotent steps. Alternative: keep reservation + inventory in the **same DB** to use ACID.

Search and pricing

- Search by location uses the geo index (see Proximity Service) or Elasticsearch; pricing may vary by day (rate table / dynamic pricing service).

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Locking	Pessimistic / Optimistic / DB constraint	Pessimistic or constraints for correctness at low QPS; optimistic for read-heavy low contention.
Database	Relational ACID / NoSQL	Relational: transactions + constraints are core to correctness.
Cross-service transaction	2PC / Saga	Saga with compensations; or co-locate reservation + inventory in one DB.
Cache for availability	Strong / advisory	Advisory cache with DB as the source of truth.

9. Scaling, reliability, failure modes

- Shard by hotel_id; replicate with failover; hold booking transactions short.
- Peak events: queue + waiting room for flash promos; rate limit; pre-warm caches.
- Archive old reservations to cold storage; keep inventory only for future dates.

10. Likely follow-up questions

- How do you handle 'hold for 15 minutes while paying'? (soft hold with expiry, status = pending + TTL job)
- How do you handle overbooking and walk-in compensation?
- How do you support dynamic pricing?
- What if the payment succeeds but the reservation write fails? (reconcile, idempotent retries, saga)

11. Common mistakes

- Using NoSQL without transactions and hoping to avoid double booking.
- Forgetting idempotency of the submit call.
- Treating the cache as the source of truth for inventory.

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"I'll assume 5,000 hotels and 1 M rooms, search plus book plus cancel, admin tools, payment handled by an external gateway, and up to 10% overbooking. The QPS is low: about 3 reservations per second on average, with spikes for promotions. So the hard part is correctness under concurrency, not throughput."
2. High-level design	10 min	"Behind an API gateway I'd have a hotel service for read-heavy data, a rate service for pricing, and a reservation service for booking and inventory, with a relational DB sharded by hotel ID. Payment is an external service. Inventory is tracked per room type per date."
3. Deep dive	25 min	"Two races. First, two users booking the last room: I solve it with a transaction that checks and updates the inventory rows, using row locks or optimistic versioning, plus a CHECK constraint as a safety net. Second, retries or double clicks: the client generates a reservation ID that is a unique key, so a duplicate insert fails and returns the same result. If services are separate, I'd use a saga with compensation; or keep reservation and inventory in one DB."
4. Wrap up	4 min	"Scaling: shard by hotel, advisory availability cache, waiting room for flash promotions. Extras: a temporary hold with an expiry during payment, dynamic pricing, and what happens if payment succeeds but the write fails (reconciliation)."

13. 60-second recap / practice checklist

- Inventory per (hotel, room type, date); reservation row with idempotent reservation_id.
- Single-DB ACID transaction with row lock / version / CHECK constraint prevents double booking; allow 10% overbook.
- Shard by hotel_id, cache for reads, saga for cross-service flows.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Stripe: Designing robust and predictable APIs with idempotency
• PostgreSQL: Explicit locking (SELECT FOR UPDATE)
• Microservices.io: Saga pattern